MINI PROJECT REPORT

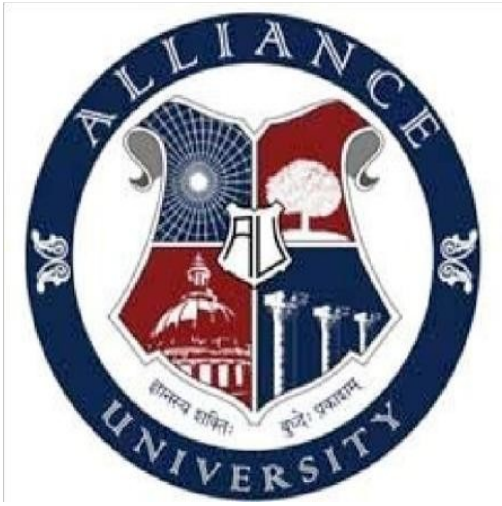
TARJAN'S STRONGLY CONNECTED COMPONENTS ALGORITHM

Submitted by:
YOUR FULL NAME
YOUR ROLL NUMBER

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
DAA FACULTY NAME
Assistant Professor, ASAC



Alliance School Of Advance Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on Tarjan's Strongly Connected Components Algorithm (RegexCPU) is submitted by YOUR FULL NAME [Uni. Roll No. YOUR ROLL NUMBER] in partial fulfilment for the award of the degree of Bachelor Of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026. This report embodies the results of original work and studies conducted by students and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

DAA FACULTY NAME
Assistant Professor



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled TARJAN'S STRONGLY CONNECTED COMPONENTS ALGORITHM, submitted by me in partial fulfilment of the requirements for the award of the degree of **Bachelor Of Technology in Computer Science and Engineering (CSE)** at **Alliance University**, is a record of my original work carried out under the supervision of **DAA FACULTY NAME, Assistant Professor, ASAC**.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

YOUR FULL NAME	YOUR ROLL NUMBER	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled TARJAN'S STRONGLY CONNECTED COMPONENTS ALGORITHM.

First and foremost, I extend my sincere thanks to my mentor and guide, DAA FACULTY NAME, for the support, insightful feedback, and encouragement throughout the development of this work. Their expertise and commitment to fostering learning provided the foundation for this project.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to implement Tarjan's algorithm iteratively, validate it against Kosaraju's algorithm and measure its linear running time on large graphs.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey.

Thank you.

YOUR FULL NAME

TABLE OF CONTENTS

Acknowledgement.....4

Abstract..... 6

Chapter I- Introduction.....7

Problem Statement.....7

Theory.....7

Chapter II - System Configuration.....9

Algorithm.....9

Source Code.....9

Chapter III - Results..... 12

Chapter IV – Conclusion..... 16

Abstract

A strongly connected component (SCC) of a directed graph is a maximal set of vertices that are all reachable from one another. Tarjan's algorithm finds all SCCs in a single depth-first traversal in O(V + E) time using low-link values. This report presents an iterative implementation of Tarjan's algorithm, an independent Kosaraju implementation used as a checker, and an application: analysing the state graph of a deterministic finite automaton to locate dead (useless) states, loops, and to decide whether the accepted language is finite.

Experiments on random graphs with up to 160,000 vertices and 480,000 edges show running time growing linearly with V + E; Tarjan took 673 ms against 859 ms for Kosaraju at the largest size. Both algorithms returned identical partitions on 300 random graphs and on every benchmark graph, and the iterative design handled a 200,000-vertex chain without recursion errors. The DAA component is part of RegexCPU, together with FLAT (regex to automata) and COA (CPU simulation).

Chapter I- Introduction

Graph traversal (BFS and DFS) is the foundation of Module 1 of Design and Analysis of Algorithms. Among the DFS-based algorithms, the computation of strongly connected components is one of the most useful: collapsing every SCC into a single vertex turns any directed graph into a directed acyclic graph (DAG), which can then be processed with topological order and dynamic programming. Tarjan's algorithm (1972) achieves this with one DFS and a stack.

Instead of demonstrating the algorithm on a textbook graph only, this project applies it to a real graph: the transition graph of a DFA produced from a regular expression. The result is used to remove states that can never lead to acceptance, to detect cycles and to classify the language as finite or infinite. The rest of the pipeline (regular expression to automaton, and execution on a simulated CPU) is described in the FLAT and COA reports.

Problem Statement

Given a directed graph with V vertices and E edges, compute all strongly connected components in linear time without recursion limits, and use them to analyse an automaton. The objectives are:

- implement Tarjan's algorithm iteratively so that deep graphs do not overflow the call stack;
- implement Kosaraju's two-pass algorithm independently and use it to cross-check Tarjan's output;
- build the condensation DAG and exploit the reverse-topological emission order of Tarjan's algorithm;
- use the SCC structure of a DFA to find dead states, cycles and language finiteness;
- verify the O(V + E) bound experimentally.

Theory

Strongly connected components

Vertices u and v are strongly connected if there is a path from u to v and from v to u. This relation is an equivalence relation; its classes are the SCCs. The **condensation graph**, with one node per SCC, is always a DAG.

Tarjan's algorithm

During a DFS each vertex v receives a discovery number `index[v]` and a `low[v]`: the smallest discovery number reachable from v's DFS subtree using at most one back or cross edge to a vertex still on the stack. Vertices are pushed on a stack when discovered. When the DFS finishes a vertex v with `low[v] = index[v]`, v is the root of an SCC and the stack is popped down to v; the popped vertices form the component.

Table 1: Properties of Tarjan's algorithm

Property	Statement
Time	O(V + E): every vertex is pushed and popped once, every edge examined once
Space	O(V) for index, low, on-stack flags and the stack
Output order	SCCs are emitted in reverse topological order of the condensation DAG
Passes	one DFS (Kosaraju needs two DFS passes and a transposed graph)

Application to a DFA

Treat each DFA state as a vertex and each transition as an edge. A state is **useful** if it is reachable from the start state and can reach an accepting state. Because Tarjan emits SCCs in reverse topological order, every successor of a component has already been emitted when the component is emitted. A single left-to-right pass can therefore compute `live[C] = (C contains an accepting state) or (some successor of C is live)`. States in non-live components are dead and are removed. The language is infinite exactly when a cyclic SCC (size > 1 or a self-loop) contains a useful state.

Chapter II - System Configuration

Table 2: Development and execution environment

Item	Configuration
Operating system	Linux 6.18.44-fc-v153
Processor	Intel(R) Xeon(R) Processor @ 2.80GHz
CPU cores	1
Memory (RAM)	3.9 GB
Language / runtime	Python 3.12.3
Libraries	Flask (web app), matplotlib + numpy (graphs); core engines use only the Python standard library
Testing	unittest (28 tests), Python `re` as reference oracle

Algorithm

Tarjan's SCC (iterative)

```
for each vertex root not yet visited:
    push (root, 0) on a work stack
    while work stack not empty:
        (v, i) = pop
        if i == 0:  index[v] = low[v] = counter++;  push v on SCC stack;  mark
on_stack[v]
        for j from i to deg(v)-1:
            w = adj[v][j]
            if w unvisited:    push (v, j+1), push (w, 0);  break      #
"recursive call"
            elif on_stack[w]:  low[v] = min(low[v], index[w])
        if we descended: continue
        if low[v] == index[v]:  pop SCC stack down to v -> one component
        if work stack not empty:  parent u = top;  low[u] = min(low[u], low[v])
```

The explicit work stack stores (vertex, next-edge-position) pairs, so the recursion of the textbook version is replaced by a loop. Python's default recursion limit of about 1,000 frames would otherwise crash on long chains.

DFA analysis

1. Run Tarjan on the complete minimal DFA graph.
2. Map every state to its component and build the condensation edges.
3. Scan components in emission order and compute `live`.
4. useful = live ∩ reachable; dead = all states − useful.
5. Report cyclic components and decide finiteness; trim dead states before code generation.

Source Code

Tarjan's algorithm (scc.py)

```
def tarjan_scc(adj):
```

```
        n = len(adj)
    index, low = [-1] * n, [0] * n
    on_stack = [False] * n
    stack, sccs, counter = [], [], 0
    for root in range(n):
        if index[root] != -1:
            continue
        work = [(root, 0)]
        while work:
            v, i = work.pop()
            if i == 0:
                index[v] = low[v] = counter
                counter += 1
                stack.append(v)
                on_stack[v] = True
            descended = False
            for j in range(i, len(adj[v])):
                w = adj[v][j]
                if index[w] == -1:
                    work.append((v, j + 1))
                    work.append((w, 0))
                    descended = True
                    break
                if on_stack[w]:
                    low[v] = min(low[v], index[w])
            if descended:
                continue
            if low[v] == index[v]:          # v is the root of an SCC
                comp = []
                while True:
                    w = stack.pop()
                    on_stack[w] = False
                    comp.append(w)
                    if w == v:
                        break
                sccs.append(comp)
            if work:                         # return to the parent: propagate
low-link
                u = work[-1][0]
                low[u] = min(low[u], low[v])
    return sccs
```

Kosaraju's algorithm (scc.py), used as a checker

```
def kosaraju_scc(adj):
        n = len(adj)
    seen, order = [False] * n, []
    for root in range(n):
        if seen[root]:
            continue
        seen[root] = True
        stack = [(root, 0)]
        while stack:
            v, i = stack.pop()
            if i < len(adj[v]):
                stack.append((v, i + 1))
                w = adj[v][i]
                if not seen[w]:
                    seen[w] = True
                    stack.append((w, 0))
            else:
                order.append(v)
    radj = [[] for _ in range(n)]
    for v in range(n):
        for w in adj[v]:
            radj[w].append(v)
    seen, sccs = [False] * n, []
    for root in reversed(order):
        if seen[root]:
            continue
        seen[root] = True
```

```
        comp, stack = [], [root]
        while stack:
            v = stack.pop()
            comp.append(v)
            for w in radj[v]:
                if not seen[w]:
                    seen[w] = True
                    stack.append(w)
        sccs.append(comp)
    return sccs
```

Condensation, dead states and finiteness (scc.py)

```
def analyze(adj, start, accepting):
        sccs = tarjan_scc(adj)
    comp_of = {}
    for cid, comp in enumerate(sccs):
        for v in comp:
            comp_of[v] = cid
    cond = [set() for _ in sccs]
    for v in range(len(adj)):
        for w in adj[v]:
            if comp_of[v] != comp_of[w]:
                cond[comp_of[v]].add(comp_of[w])
    # Tarjan order = reverse topological: every successor SCC has a SMALLER id.
    live_comp = [False] * len(sccs)
    for cid, comp in enumerate(sccs):
        live_comp[cid] = any(v in accepting for v in comp) or any(live_comp[d] for
d in cond[cid])
    live = {v for v in range(len(adj)) if live_comp[comp_of[v]]}
    reachable = reachable_from(adj, start)
    useful = live & reachable
    cyclic = [cid for cid, comp in enumerate(sccs)
              if len(comp) > 1 or comp[0] in adj[comp[0]]]
    infinite = any(any(v in useful for v in sccs[cid]) for cid in cyclic)
    return {
        "sccs": sccs,
        "comp_of": comp_of,
        "condensation": sorted((a, b) for a in range(len(sccs)) for b in cond[a]),
        "topo_order": list(range(len(sccs) - 1, -1, -1)),
        "reachable": reachable,
        "live": live,
        "useful": useful,
        "dead": set(range(len(adj))) - useful,
        "cyclic_sccs": cyclic,
        "is_infinite": infinite,
    }
```

Chapter III - Results

Sample input and output

Input: the regular expression `ab|cd?`. Its minimal complete DFA has five states: start, two accepting states and a trap state. The command `python main.py "ab|cd?" --scc -t ab cd abc` produced:

```
Pattern            : ab|cd?
Postfix            : a b . c d ? . |
Alphabet           : ['a', 'b', 'c', 'd']
epsilon-NFA        : 12 states, 13 edges
DFA (subset)       : 5 states
Complete DFA       : 6 states
Minimal DFA        : 5 states
Tarjan SCCs        : 5 (largest 1, cyclic 1)
Dead states pruned : [2]
Language           : finite -> ['c', 'ab', 'cd']
CPU program        : 17 code words + 20 data words

Tarjan output (reverse topological order of the condensation DAG):
  SCC 0: states [2] cyclic DEAD
  SCC 1: states [4] accepting
  SCC 2: states [1]
  SCC 3: states [3] accepting
  SCC 4: states [0]
Condensation edges: [(1, 0), (2, 0), (2, 1), (3, 0), (3, 1), (4, 0), (4, 2), (4, 3)]
Dead states: [2]

input             NFA    DFA    minDFA  CPU     instr  cycles  CPI    agree
'ab'              ACC    ACC    ACC     ACC     30     157     5.233  OK
'cd'              ACC    ACC    ACC     ACC     30     157     5.233  OK
'abc'             rej    rej    rej     rej     34     179     5.265  OK
```

Table 3: Tarjan output for the DFA of ab|cd? (reverse topological order)

SCC	States	Flags
0	q2	cyclic DEAD
1	q4	accepting
2	q1	–
3	q3	accepting
4	q0	–

The only cyclic component is the trap state q2 with its self-loop; it cannot reach an accepting state, so Tarjan's pass marks it dead and it is removed. All other components are singletons without cycles, so the language is finite, and the enumeration `c, ab, cd` is printed. For `(a|b)*abb` the whole four-state DFA is a single cyclic SCC, so the language is infinite.

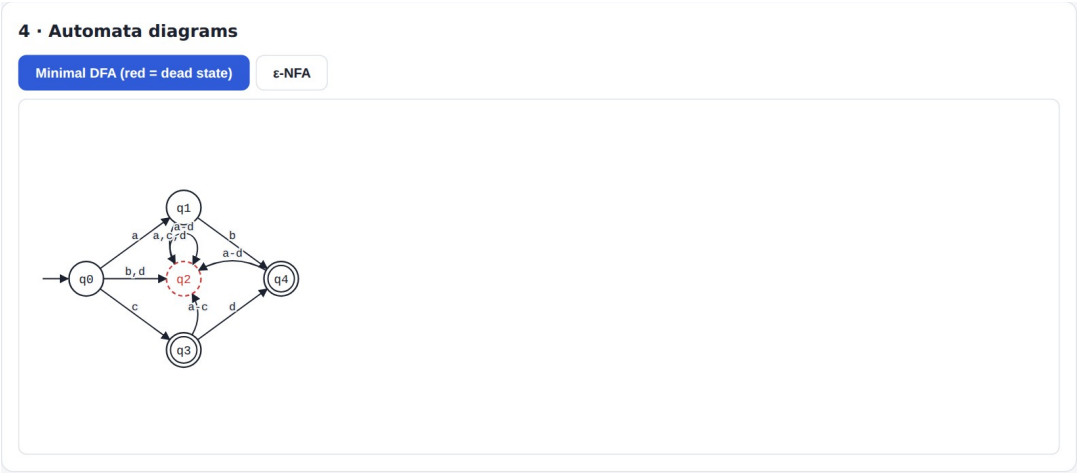


Figure 1: Minimal DFA of ab|cd? with the dead state q2 drawn in red

5 · Tarjan SCC analysis (DAA)

SCC	states	flags
0	q2	cyclic dead
1	q4	accepting
2	q1	–
3	q3	accepting
4	q0	–

SCCs are listed in the order Tarjan emits them – reverse topological order of the condensation DAG.

Figure 2: Web application: SCC table for ab|cd?

Complexity analysis

Table 4: Running time on random graphs with 3 outgoing edges per vertex (best of 3 runs)

V	E	V + E	SCCs found	Tarjan (ms)	Kosaraju (ms)
2,000	6,000	8,000	128	2.4	2.3
5,000	15,000	20,000	299	10.0	6.6
10,000	30,000	40,000	571	17.5	16.6
20,000	60,000	80,000	1135	52.3	48.7
40,000	120,000	160,000	2366	106.5	137.7
80,000	240,000	320,000	4753	324.1	393.7
160,000	480,000	640,000	9410	673.0	859.2

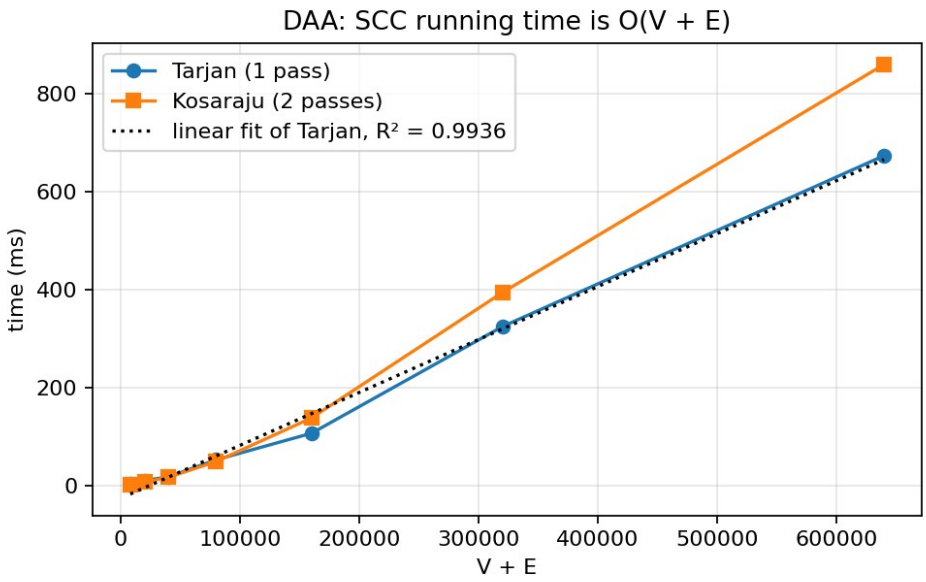


Figure 3: Measured running time against V + E with linear fit

Doubling V + E roughly doubles the running time for both algorithms, as predicted by the O(V + E) analysis; the linear fit of Tarjan's times has R^2 close to 0.99. In this Python implementation Tarjan and Kosaraju have similar constants: Tarjan is faster at the largest sizes because it makes a single pass and builds no transposed graph, but at small sizes the difference is within measurement noise, so no stronger claim is made. Absolute timings depend on the machine.

Test case results

Table 5: Automated test results (python -m unittest)

Test group	Tests	Result
NFA / DFA / minimisation	5	PASS
CPU, assembler, instruction cycle	7	PASS
End-to-end vs Python `re` (incl. 3,000-case fuzz)	4	PASS
Regex parser (postfix, classes, error handling)	5	PASS
Tarjan & Kosaraju SCC, dead states, finiteness	7	PASS
Total	28	**ALL PASS**

Table 6: Selected DAA test cases (tests/test_all.py, class TestSCC)

Test case	Expected	Result
Textbook graph with SCCs {0,1,2}, {3,4,5}, {6,7}, {8}	4 components	PASS
Chain 3→0→1→2: emission order	2, 1, 0, 3 (reverse topological)	PASS
Self-loop and isolated vertex; empty graph	[[0],[1]]; []	PASS
300 random graphs, Tarjan vs Kosaraju	identical partitions	PASS

Path graph with 200,000 vertices	200,000 components, no RecursionError	PASS
DFA of ab*: dead states	exactly the trap state	PASS
a* infinite; ab|cd? finite, words c, ab, cd	as stated	PASS

Chapter IV – Conclusion

Tarjan's algorithm was implemented iteratively, verified against an independent Kosaraju implementation and shown experimentally to run in time linear in V + E. Applied to the DFA produced by the FLAT part of the project, it identifies the dead trap state, detects loops and decides whether a language is finite, all from one traversal and one pass over the condensation DAG. The pruned automaton is then compiled to machine code and run on the simulated CPU of the COA part.

The key idea worth remembering is the reverse-topological emission order: it lets dynamic programming over the condensation DAG run during the SCC computation itself. Possible extensions are bridge and articulation-point detection with the same low-link technique, 2-SAT solving, and analysing larger automata such as lexer DFAs.